CS 300 – Chapter 9 Notes

Balanced Search Trees: AVL and Red-Black

Jose de Lima

April 21, 2026

1 9.1 AVL: The Balanced BST

Chapter 6 laid out the dream (BSTs as fast ordered maps) and the nightmare (sorted input $\Rightarrow O(n)$ operations). Chapter 7.5 showed a *probabilistic* fix with treaps. This chapter does the rigorous, deterministic version. The structure: an **AVL tree**, named for its 1962 inventors Adelson-Velsky and Landis – the first self-balancing BST ever published.

Definition — AVL tree

An AVL tree is a BST that additionally satisfies the *height-balance property*: for every node, the heights of its left and right subtrees differ by at most 1. Equivalently, for every node the **balance factor**

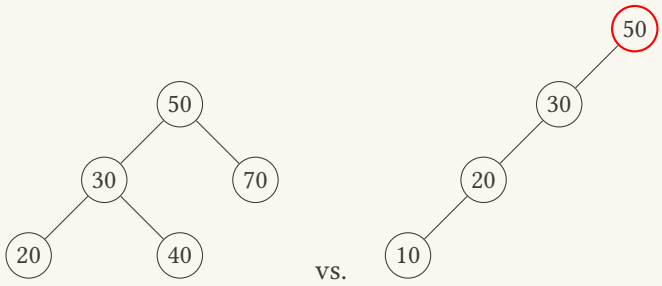
$$\text{bf}(n) = \text{height}(n.\text{left}) - \text{height}(n.\text{right})$$

is in $\{-1, 0, +1\}$.

Height is measured the usual way (number of edges on the longest root-to-leaf path in the subtree), with one convention: a *null* (absent) subtree has height -1. That makes a single-node tree have height 0 (its children are both null with height -1; $1 + \max(-1, -1) = 0$). Same convention as section 6.8.

Is it an AVL tree? Check every node.

You can't eyeball AVL-ness by looking at the tree globally – you check the balance factor at every node. A single node whose balance factor is ± 2 is enough to disqualify the whole tree.



Left: every node's two subtree heights differ by ≤ 1. *AVL.*

Right: node 50 has left-subtree height 2 and right-subtree height -1 (null). Balance factor $= 2 - (-1) = 3$. *Not AVL.* Even though 30 and 20 would look locally acceptable, the root violates the property.

Height guarantee

Why does this restriction matter? Because it forces $h = O(\log n)$.

Definition — Fibonacci bound on AVL height

An AVL tree with n nodes has height $h \leq 1.44 \log_2(n+2)$ in the worst case – about 1.44 times the theoretical minimum. In practice it's even tighter.

The proof uses the minimum-node AVL tree at each height: the smallest AVL of height h has $N(h)$ nodes where $N(h) = N(h-1) + N(h-2) + 1$, the Fibonacci recurrence. Inverting this gives $h \leq 1.44 \log_2 n + O(1)$. The 1.44 constant is the price you pay for AVL-balanced instead of perfectly-balanced – cheap, and well worth it since perfect balance is unreachable during live insertion/removal.

Compare:

n	Min height (perfect)	AVL max height
1 000	9	14
1 000 000	19	28
10^9	29	43

So even in the worst case, an AVL tree operation on a billion keys costs at most 43 comparisons – still emphatically $O(\log n)$, not even close to the raw BST's potential 10^9.

Storing height at each node

AVL algorithms need to compute balance factors cheaply. Computing a subtree's height by recursively walking it is $O(n)$ – unaffordable when insert/remove has to check balance factors at every ancestor. The fix: **cache the height at each node.**

```
template <typename T>
struct AVLNode {
    T          data;
    AVLNode*   left   = nullptr;
    AVLNode*   right  = nullptr;
    AVLNode*   parent = nullptr;   // usually wanted for retrace (section 9.3)
    int        height = 0;         // height of THIS subtree; null children =
        -1
};

template <typename T>
int height(const AVLNode<T>* n) { return n ? n->height : -1; }

template <typename T>
int balanceFactor(const AVLNode<T>* n) {
    return height(n->left) - height(n->right);
}
```

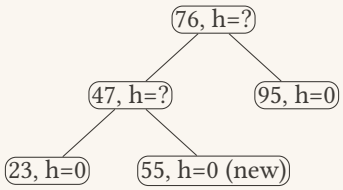
With height stored in the node, balance factor is $O(1)$. The cost: after every structural change (insertion, rotation, removal) we must *recompute the height of every affected ancestor.* This recomputation is why AVL needs parent pointers (or an iterative walk back up from the modified node).

Updating heights after a change

When node X changes (gains a child, loses one, is rotated), the heights of X and every ancestor up to the root may change. Update rule:

$$\text{height}(X) = 1 + \max(\text{height}(X.\text{left}), \text{height}(X.\text{right}))$$

Apply from X upward toward the root. As soon as you reach an ancestor whose height didn't change, you can stop – the heights further up are unaffected.



Insert 55 under 47. After the insertion: 55 is a leaf, $h = 0$. At 47: $\max(0, 0) + 1 = 1$. At 76: $\max(1, 0) + 1 = 2$. Balance factors: 47 has 0, 76 has $1 - 0 = 1$. Still $\in \{-1, 0, +1\}$ everywhere. Still AVL.

Key idea

The stored-height trick is what makes AVL practical. Without it, every insert would need an $O(n)$ rescan of subtree heights to verify balance. With it, the per-insert rebalancing cost stays $O(\log n)$ because you only touch ancestors on one root-to-leaf path.

Why this invariant, and not something else?

The "differ by at most 1" constraint is a specific compromise:

- **Stricter** ("differ by at most 0" = perfectly balanced): impossible to maintain under arbitrary insertion, because you can't turn $N = 3$ into $N = 4$ with perfect symmetry.
- **Looser** ("differ by at most 2" or more): allows more imbalance before triggering rebalancing, but height grows faster and search slows down.
- ± 1: tightest strictness that can be maintained with a constant number of local rotations after each modification. It's the sweet spot.

Red-black trees (section 9.8, sort of – the main chapter focuses on AVL) use a weaker balance invariant that allows larger height differences but cheaper rebalancing. They're strictly less balanced than AVL (worst-case height $\sim 2 \log n$ vs. AVL's $1.44 \log n$), but their rebalancing has lower overhead on average. `std::map` uses red-black; competitive programmers and strict-performance workloads sometimes prefer AVL.

Road map for chapter 9

Sections 9.2–9.7 formalize rotations (the single primitive AVL uses to fix violations), give the full insert/remove algorithms, and walk through the four rebalancing cases (left-left, left-right, right-right, right-left). This section established *what* AVL is; the rest of the chapter is *how*.

Gotcha

Don't conflate AVL's balance factor with red-black's node colors. Both are auxiliary per-node state used to maintain structural invariants, but they encode different constraints and trigger different rebalancing rules. AVL says "left minus right height is in $\{-1, 0, +1\}$." Red-black says "no red node has a red child, and every path from root to any null has the same number of black nodes." Different invariants, different rotations, different use cases.

2 9.2 Rotations: The Only Primitive AVL Needs

Section 9.1 said "we restore balance with local rotations." This section defines the rotation precisely and gives all the helper procedures AVL operations will need. One operation – the **rotation** – is the whole mechanical toolkit. AVL insert, AVL remove, and AVL rebalance all boil down to "figure out which rotation to apply, then apply it."

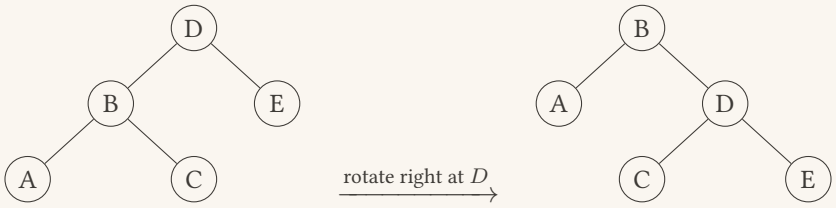
Definition — Rotation

A rotation is a local rearrangement of a BST that *changes the shape* of a subtree while *preserving the in-order (BST) ordering* of its keys. Every other tree invariant (heights, parent pointers) must be updated by the rotation code. BST ordering is the one property that survives automatically.

Rotations were teased in chapter 7.5 with treaps – same operation, different trigger. In a treap the trigger is "a child has a higher priority than its parent." In an AVL tree the trigger is "this node's balance factor is ± 2." The rotation primitive is identical in both.

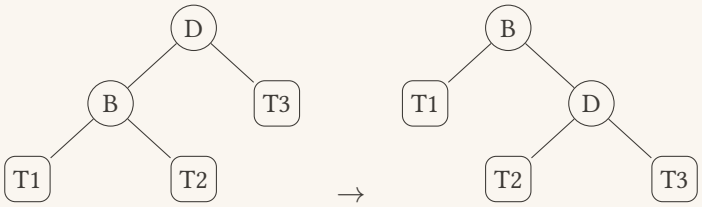
Right rotation

A **right rotation at node** D requires D to have a left child B. After the rotation, B is the new subtree root and D becomes B's right child. B's former right child (call it C) becomes D's new left child – that's the move that preserves BST ordering.



Ordering check: before the rotation, the in-order sequence is A, B, C, D, E. After: A, B, C, D, E. Identical – rotations are *identity on in-order sequence*. Height behavior: if the original tree was left-heavy (left subtree taller), the rotation reduces the height difference; that's why this fixes AVL violations on the left side.

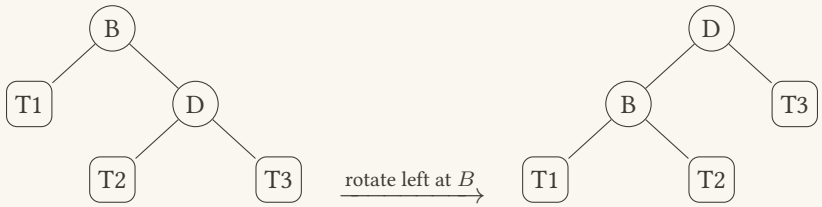
The generalization to subtrees (not just single nodes) is what makes rotations useful for arbitrary tree sizes:



T_1, T_2, T_3 are arbitrary subtrees. The rotation only rewires three pointers at the top; the subtrees ride along unchanged. That's why each rotation is $O(1)$.

Left rotation

The mirror image: a **left rotation at node** D requires D to have a right child. It promotes the right child to the subtree root.



A left rotation is literally the inverse of the matching right rotation.

Utility: update-height, set-child, replace-child, balance-factor

Before showing rotation code, we need the housekeeping helpers. These all operate on `AVLNode<T>` from section 9.1 (with cached `height` and `parent` fields).

```
template <typename T>
void avlUpdateHeight(AVLNode<T>* n) {
    int lh = (n->left  != nullptr) ? n->left->height  : -1;
    int rh = (n->right != nullptr) ? n->right->height : -1;
    n->height = 1 + std::max(lh, rh);
}

template <typename T>
int avlBalance(const AVLNode<T>* n) {
    int lh = (n->left  != nullptr) ? n->left->height  : -1;
    int rh = (n->right != nullptr) ? n->right->height : -1;
    return lh - rh;
}

// Set `child` as parent's left/right. Fix child's parent pointer.
// Recompute parent's cached height.
template <typename T>
void avlSetChild(AVLNode<T>* parent, bool isLeft, AVLNode<T>* child) {
    if (isLeft) parent->left = child;
    else        parent->right = child;
    if (child != nullptr) child->parent = parent;
    avlUpdateHeight(parent);
}

template <typename T>
void avlReplaceChild(AVLNode<T>* parent,
                     AVLNode<T>* currentChild,
                     AVLNode<T>* newChild) {
    if (parent->left == currentChild)  avlSetChild(parent, true,  newChild);
    else                               avlSetChild(parent, false, newChild);
}
```

`avlSetChild` does the *four* things you always forget together: (1) wire parent's child slot, (2) wire child's parent pointer, (3) the caller picks which slot, (4) recompute height. Any AVL implementation that tries to inline these will eventually get them out of sync and ship a bug.

Right rotation algorithm

```
template <typename T>
AVLNode<T>* avlRotateRight(AVLNode<T>*& root, AVLNode<T>* D) {
    AVLNode<T>* B  = D->left;          // must be non-null
    AVLNode<T>* BR = B->right;         // "C" in the diagram -- may be null

    if (D->parent != nullptr) avlReplaceChild(D->parent, D, B);
    else                      { root = B; B->parent = nullptr; }

    avlSetChild(B, false, D);          // B's new right is D
    avlSetChild(D, true,  BR);         // D's new left is B's old right
    return B;                          // B is the new subtree root
}
```

Three rewires at the top of the subtree, heights recomputed as part of each `setChild`, and we return the new root so callers can continue walking upward if needed.

Left rotation algorithm

Symmetric – swap every "left" for "right":

```
template <typename T>
AVLNode<T>* avlRotateLeft(AVLNode<T>*& root, AVLNode<T>* B) {
    AVLNode<T>* D  = B->right;
    AVLNode<T>* DL = D->left;

    if (B->parent != nullptr) avlReplaceChild(B->parent, B, D);
    else                      { root = D; D->parent = nullptr; }

    avlSetChild(D, true,  B);
    avlSetChild(B, false, DL);
    return D;
}
```

The rebalance procedure

After any insertion or removal, walk from the modified node up to the root; at each ancestor, update its cached height and check its balance factor. If the balance factor is ± 2, apply one or two rotations to fix it. This is the procedure. All four cases (left-left, left-right, right-right, right-left) flow through it:

```
template <typename T>
AVLNode<T>* avlRebalance(AVLNode<T>*& root, AVLNode<T>* n) {
    avlUpdateHeight(n);
    int bf = avlBalance(n);

    if (bf == 2) {                              // left-heavy
        if (avlBalance(n->left) == -1)          // left-right case: pre-rotate
            avlRotateLeft(root, n->left);
        return avlRotateRight(root, n);
    }
    if (bf == -2) {                             // right-heavy
```

```
        if (avlBalance(n->right) == 1)          // right-left case: pre-rotate
            avlRotateRight(root, n->right);
        return avlRotateLeft(root, n);
    }
    return n;                                   // -1, 0, or 1: already
        balanced
}
```

The two inner checks detect the *double-rotation cases*. If the left child's balance factor has the opposite sign from the parent's (i.e., $\mathrm{bf}(n.\mathrm{left}) = -1$ when $\mathrm{bf}(n) = +2$), a single right rotation at n would not fix the imbalance – we first left-rotate n's left child to turn a "zig-zag" into a "zig-zig", then right-rotate n. These are the four cases, collapsed into six lines of code.

The four imbalance cases

Name	Trigger (balance factors)	Fix
Left-Left	$\mathrm{bf}(n) = +2$, $\mathrm{bf}(n.\mathrm{left}) \geq 0$	single right-rotate at n
Left-Right	$\mathrm{bf}(n) = +2$, $\mathrm{bf}(n.\mathrm{left}) = -1$	left-rotate at n.left, then right-rotate at n
Right-Right	$\mathrm{bf}(n) = -2$, $\mathrm{bf}(n.\mathrm{right}) \leq 0$	single left-rotate at n
Right-Left	$\mathrm{bf}(n) = -2$, $\mathrm{bf}(n.\mathrm{right}) = +1$	right-rotate at n.right, then left-rotate at n

Left-Left and Right-Right are the "easy" cases: the imbalance is already aligned and a single rotation fixes it. Left-Right and Right-Left are the "zig-zag" cases: the imbalance is shaped like a kink in the path from parent to grandchild, and you first rotate to straighten the kink, then rotate again to fix the height.

Key idea

Notice the elegance: two rotation primitives (left, right), four named cases (LL, LR, RR, RL), but the rebalance procedure decides which to apply with a single check on the balance factor of the child along the heavy side. You don't name cases in code – the balance factors pick the right rotation(s) automatically.

Gotcha

Every rotation and setChild updates cached heights *locally*. The caller is responsible for continuing the retrace – walking up to the next ancestor and calling `avlRebalance` there too, because a rotation at a child can still leave an ancestor further up imbalanced. Section 9.3 (or wherever the insert algorithm lives) shows that loop.

Why rotations alone are sufficient

It's worth marveling at: every AVL rebalancing is one or two rotations. The shape adjustment you need to fix an imbalance is always local – contained within a grandparent-parent-child trio – because AVL has such a tight invariant that violations can't be more than height-two deep before they're caught. This is what makes AVL fast: worst-case two rotations per insert, one rotation per removal iteration. All $O(1)$ per fix; walking back up the tree is $O(\log n)$, giving $O(\log n)$ total per modification.

3 9.3 AVL Insertion

Section 9.2 gave us the rotation primitives; this section assembles them into a full insertion algorithm. The structure is simple to state and slightly fiddly to implement:

Definition — AVL insertion

1. **Insert** the new node using the standard BST insertion (walk down, attach at the first empty child slot).
2. **Retrace** upward from the new node's parent. At each ancestor, update the cached height and check the balance factor.
3. **Rebalance** on the first node whose balance factor is ± 2. Apply one or two rotations; this restores the invariant for the entire path below. Continue retracing because a rotation can change the subtree height, which can expose further imbalance higher up.

Retrace: what changes when

Inserting a leaf L increases the height of some ancestors by 1 but not others. Specifically, the height of L's parent P becomes $h'_P = \max(h_P, 0) + 1$. If P previously had a non-null sibling child at the same height, P's height doesn't change – the new node just filled the other slot. If P was a leaf before, P's height increased from 0 to 1, and that change propagates upward.

This cascading-height behavior is why we walk all the way up: we don't know in advance how far the height increase propagates. We stop propagating (for height-update purposes) at the first ancestor whose height didn't change. But for balance-factor checking we still walk all the way up – even if heights stabilize, a descendant rotation can shift things.

The canonical recipe is simpler in code: "walk to root, call `avlRebalance` at every ancestor." `avlRebalance` updates height and checks balance factor on its own, so a single procedure does both jobs.

The four insertion cases (by imbalance shape)

After inserting, the first unbalanced ancestor P (balance factor ± 2) is fixed by one of four cases, named for the direction to the grandchild where the insertion happened:

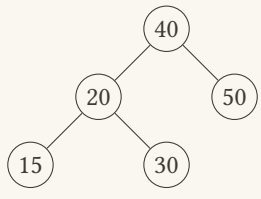
Shape	Signal	Fix
Left-Left	$\text{bf}(P) = +2$, $\text{bf}(P.\text{left}) \geq 0$	right-rotate at P
Left-Right	$\text{bf}(P) = +2$, $\text{bf}(P.\text{left}) = -1$	left-rotate at P.left, then right-rotate at P
Right-Right	$\text{bf}(P) = -2$, $\text{bf}(P.\text{right}) \leq 0$	left-rotate at P
Right-Left	$\text{bf}(P) = -2$, $\text{bf}(P.\text{right}) = +1$	right-rotate at P.right, then left-rotate at P

Left-Left and Right-Right are "straight" imbalances: the insertion went deeper on an already-heavy side, and a single rotation fixes the heights. Left-Right and Right-Left are "zig-zag" imbalances where the first rotation *straightens* the shape into a Left-Left or Right-Right, and the second rotation then fixes it.

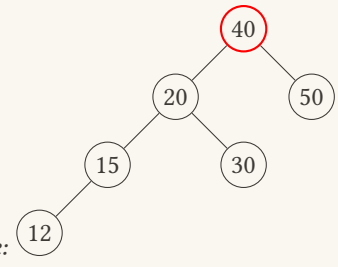
Left-Left example

Insert 12 into this tree:

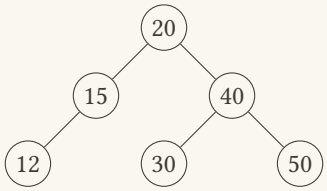
Before:



→ *after insert 12, before rebalance:*



After insertion, height of 20 is 2, height of 50 is 0. $\mathrm{bf}(40) = 2 - 0 = 2$. Problem. $P = 40$, $P.\mathrm{left} = 20$, $\mathrm{bf}(20) = 1 - 0 = 1 \geq 0$: **Left-Left case**. Single right-rotate at 40:

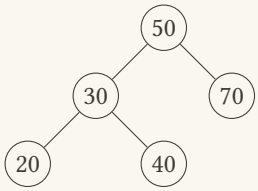


Every balance factor is now in $\{-1, 0, +1\}$. Tree is back to AVL in one rotation.

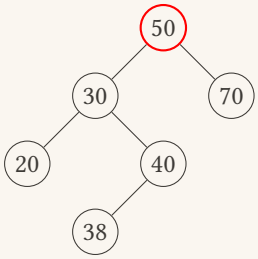
Left-Right example (the double-rotation case)

Insert 38 into this tree:

Before:



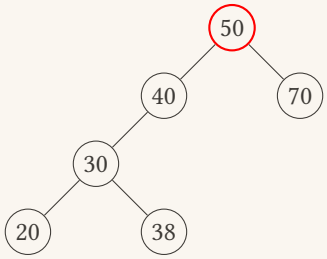
→ *after insert 38:*



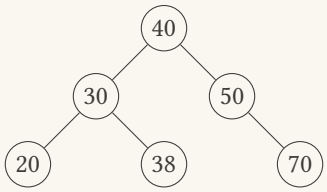
$\mathrm{bf}(50) = 2$, $\mathrm{bf}(30) = -1$: **Left-Right case**. Two-step fix:

1. Left-rotate at 30 (the left child). This straightens the L-R zig-zag.
2. Right-rotate at 50.

After step 1 (left-rotate at 30):



Now the shape is Left-Left ($\mathrm{bf}(50) = 2$, $\mathrm{bf}(40) = 1$). Step 2, right-rotate at 50:



Balanced. Right-Right and Right-Left are the mirror images.

The insertion algorithm, with retrace

```
template <typename T>
void avlInsert(AVLNode<T>*& root, AVLNode<T>* node) {
    node->left = node->right = node->parent = nullptr;
    node->height = 0;

    if (root == nullptr) { root = node; return; }

    // Phase 1: standard BST insertion.
    AVLNode<T>* cur = root;
    while (true) {
        if (node->data < cur->data) {
            if (cur->left == nullptr) {
                cur->left = node; node->parent = cur; break;
            }
            cur = cur->left;
        } else {
            if (cur->right == nullptr) {
                cur->right = node; node->parent = cur; break;
            }
            cur = cur->right;
        }
    }

    // Phase 2: retrace. Walk from the parent up to the root,
    // rebalancing each ancestor.
    for (AVLNode<T>* n = node->parent; n != nullptr; n = n->parent) {
        avlRebalance(root, n);
        // After a rotation, n->parent might have changed. Re-read via the
        // node that came out of rebalance; but if rebalance returned the
        // new root, we want to keep going via its parent pointer. The
        // for-loop update reads n->parent, so this works as-is because
        // setChild/rotations keep parent pointers consistent.
    }
}
```

Two passes down-then-up, both $O(\log n)$. Inserting a duplicate is handled by the "$>=$" branch going right – equal keys end up in the right subtree.

Cost

Phase	Cost
BST descent	$O(\log n)$ (height is AVL-bounded)
Retrace	$O(\log n)$ ancestors visited
Rebalance each	$O(1)$ per ancestor (at most 2 rotations total)
Total	$\mathbf{O}(\log \mathbf{n})$

Space is $O(1)$ auxiliary (a handful of temporaries for the rotation helpers). Compare this to the raw BST insert in section 6.5: identical descent cost, identical insertion cost, plus an $O(\log n)$ retrace that guarantees the tree never degrades. A small constant-factor tax for a permanent worst-case guarantee.

Why the "first unbalanced ancestor" is enough

An important theorem (not proved in the textbook but worth knowing): *during AVL insertion, fixing the first unbalanced ancestor restores the AVL property everywhere.* Higher ancestors *will* have their cached heights touched during the retrace, but their balance factors will fall back into $\{-1, 0, +1\}$ automatically because a rotation at the first bad ancestor reduces that subtree's height by 1, undoing the height increase the insertion caused.

This is why insertion needs *at most two rotations* total (one pair for the Left-Right or Right-Left case, one for Left-Left or Right-Right). Insertion is cheaper than removal in AVL, which in the worst case may require $O(\log n)$ rotations along the path.

Key idea

AVL insert is "BST insert, then walk back up fixing things." That pattern – descend with intent, ascend to repair – shows up again in red-black insert, in B-tree split-on-insert, and in persistent data structures. Every self-balancing structure makes the descent cheap, stores some auxiliary state at each node, and pays the $O(\log n)$ ascent to reconcile after modification.

Phone book: why AVL matters

The motivating example is a phone book loaded from already-sorted names. In a raw BST this is the pathological case – height grows to $N - 1$. In an AVL tree, each insertion may trigger a rotation, and after loading N sorted keys you end up with height $\approx \log_2 N$, even though no permutation-shuffling happened. The AVL invariant silently transforms a degenerate input into a balanced structure, at a cost of one or two rotations per insert.

Gotcha

Be careful about equal-key handling. The code above sends equal keys to the right subtree. This is consistent but not *stable* (insertion order among equal keys isn't preserved). For applications that need stable ordering, store a tiebreaker (like insertion sequence number) in the comparison key, or use a `multiset`-style structure that tracks insertion order separately.

4 9.4 AVL Removal

Removal is where AVL earns a slightly worse worst-case-rotation count than insertion. For insertion, at most one cascade (one single or one double rotation) handles everything. For removal, a single rotation can *shorten* a subtree, which can trigger another imbalance one level up, which can trigger another, and so on –

up to $O(\log n)$ rotations along the retrace path.

Definition — AVL removal

1. **BST-remove** the node using the standard four-case algorithm from chapter 6: leaf, single-child, two-children (copy-successor-then-remove), root-specialization.
2. **Retrace** upward from the parent of the physically-removed node. At each ancestor, call `avlRebalance` to update the cached height and apply 0, 1, or 2 rotations as needed.
3. Continue retracing to the root. Unlike insertion, you *cannot* stop at the first fix – removal-induced height decreases can propagate arbitrarily far.

The two-children shortcut

Zchapter 6.6 showed that removing a node with two children is handled by copying the in-order successor's data over the doomed node, then recursively removing the successor. That trick still works for AVL – the successor always has at most one child (it's the leftmost of the right subtree), so the recursive call lands in the leaf / one-child case and rebalances from there. Recursion handles the retrace; you don't need to rebalance twice.

The algorithm

```
template <typename T>
bool avlRemoveNode(AVLNode<T>*& root, AVLNode<T>* node) {
    if (node == nullptr) return false;

    // Case 1: two children -- copy successor's data, recurse to remove it.
    if (node->left != nullptr && node->right != nullptr) {
        AVLNode<T>* succ = node->right;
        while (succ->left != nullptr) succ = succ->left;
        node->data = succ->data;
        return avlRemoveNode(root, succ);
    }

    AVLNode<T>* parent = node->parent;
    AVLNode<T>* only   = (node->left != nullptr) ? node->left : node->right;

    // Case 2: node is the root (0 or 1 child).
    if (node == root) {
        root = only;
        if (root != nullptr) root->parent = nullptr;
    }
    // Cases 3 & 4: splice `only` (possibly null) into node's slot below
       parent.
    else {
        avlReplaceChild(parent, node, only);
    }

    delete node;

    // Retrace: walk up from parent to root, rebalancing every ancestor.
    for (AVLNode<T>* n = parent; n != nullptr; n = n->parent) {
        avlRebalance(root, n);
    }
```

```
    return true;
}

template <typename T>
bool avlRemoveKey(AVLNode<T>*& root, const T& key) {
    AVLNode<T>* node = bstSearch(root, key);
    return avlRemoveNode(root, node);
}
```

Notice the retrace walks *all the way* to the root. For insertion, we could stop at the first rebalanced ancestor; for removal, we can't. A single rotation might fix one level and shorten the subtree, which exposes a new imbalance at the next level up.

Why removal can cascade

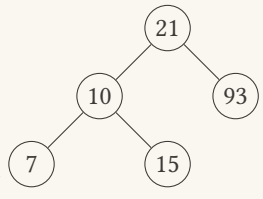
Consider a narrow AVL tree where a rebalancing rotation reduces a subtree's height by 1. That subtree's parent, which was balanced before, might now see $\text{bf} = \pm 2$ and need its own rotation. That rotation might again reduce height by 1 and propagate.

In the worst case this cascade runs the entire height of the tree, so $O(\log n)$ rotations. Each rotation is $O(1)$, so total cost is $O(\log n)$. Still logarithmic; just a slightly worse constant than insertion.

Walk-through: simple rotation after removal

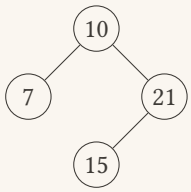
Start with this AVL tree and remove node 93:

Before:



After removing 93 (case 1 – leaf), the right subtree of 21 is null. Height of 21's left subtree is 1, right is -1. $\text{bf}(21) = 1 - (-1) = 2$. **Left-Left imbalance** ($\text{bf}(10) = 0$ – note this counts as Left-Left because the condition on the child is $\text{bf} \geq 0$). Right-rotate at 21:

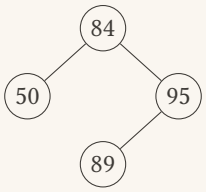
After:



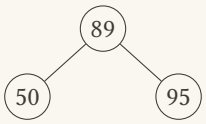
All balance factors in $\{-1, 0, +1\}$. Done.

Walk-through: successor case

Remove 84 from:



Node 84 has two children. Successor = leftmost of right subtree = 89. Copy 89's data over 84, then recursively remove 89 (which is now a leaf at its original position):



Retrace from 95 (where 89 was attached): $\text{bf}(95) = -1 - (-1) = 0$. $\text{bf}(89, \text{the new root}) = 0 - 0 = 0$. No rotations. Tree is balanced. Notice: the successor copy only moves data, not the node itself; all parent/child relationships stay intact below.

Cost

Phase	Cost
BST search + remove	$O(\log n)$
Retrace	$O(\log n)$ ancestors
Rotations along retrace	Up to $O(\log n)$ total (each $O(1)$)
Total	$\mathbf{O}(\log \mathbf{n})$

Still $O(\log n)$. The worst-case *rotation count* is higher than insertion's (up to $O(\log n)$ instead of at most 2), but each rotation is still constant work and the retrace path is the same length.

Key idea

Insertion gets a rebalancing "freebie" because a single rotation restores the original pre-insertion height – propagation stops. Removal doesn't: a rotation reduces the subtree's height by one, which is the whole problem being cascaded upward. This asymmetry is fundamental to self-balancing trees; red-black trees have it too, expressed as "black-height decreased by one" instead of "bf wrong."

Implementation pitfalls

Gotcha

When rebalancing after removal, you must walk all the way to the root. Do not stop at the first rotated ancestor. If you do, you may leave a ± 2 imbalance further up that will corrupt subsequent operations. The `for` loop `n = n->parent` is the whole fix – as long as it runs until `n == nullptr`, you're good.

Gotcha

The order "delete the node, then retrace from parent" is important. If you retrace from the node itself, you'll dereference freed memory. Always save the parent pointer before the deletion.

Aside

In production codebases that use AVL, this is where the bug density lives. The four-case BST-removal logic has two mutation orders (case 1 recurses, cases 2–4 splice), parent-pointer bookkeeping is easy to fumble, and "retrace all the way" is easy to forget. Build a test suite that hammers insert/remove pairs and checks the AVL invariant after every operation – the bug surface is too big for "it looked right" to be enough.

5 9.5 Red-Black Trees: A Second Balanced BST

Sections 9.1–9.4 covered AVL. Sections 9.5–9.7 cover **red-black trees** – the other famously self-balancing BST, and the one used by `std::map`, `std::set`, Java's `TreeMap`, the Linux kernel's completely-fair scheduler, and most production ordered-map structures.

Both AVL and red-black trees solve the same problem (guaranteed $O(\log n)$ operations on a BST despite arbitrary input). They differ in:

- **The invariant.** AVL uses a strict height-balance rule; red-black uses a looser color/black-height rule.
- **The rebalancing cost.** Red-black typically needs fewer rotations per insert/remove on average, at the cost of slightly taller trees.
- **Who uses them.** Red-black has won in the standard libraries; AVL is more common in competitive programming and legacy academic code.

Definition — Red-black tree

A red-black tree is a BST where every node has a *color* of either **red** or **black**, subject to five rules:

1. Every node is colored red or black.
2. The root is black.
3. A red node cannot have a red child.
4. Every null child is treated as a black leaf ("sentinel").
5. Every root-to-null path passes through the same number of black nodes.

Together, these rules force the longest root-to-leaf path to be at most twice the shortest, giving $h \leq 2\log_2(n+1)$.

The five rules, one by one

Rule 1: every node is red or black. Colors are auxiliary state, like AVL's height field. They're usually stored as a single bit in each node.

Rule 2: the root is black. A pure convenience – makes some cases easier. If a rebalancing step leaves the root red, you recolor it black.

Rule 3: no red node has a red child. This is the core "can't cluster" rule. Red nodes are "extra" nodes slotted between black nodes; two reds in a row would distort the balance.

Rule 4: null children count as black leaves. These are the "nil" sentinels. Some implementations create one shared nil node with color black and point every null child at it; others just treat `nullptr` as "black."

Rule 5: equal black-height on every root-to-null path. This is the balance invariant. Define **black-**

height $\mathrm{bh}(n)$ as the number of black nodes on the path from n down to any descendant null sentinel, counting the sentinel but not n itself. Rule 5 says bh is well-defined (doesn't depend on which path you pick).

Why five rules?

Rules 3 and 5 together do the real work. Rule 3 prevents red clusters, so any path has alternating reds and blacks in the worst case. Rule 5 pins down the count of blacks. Combined, the longest path (alternating red-black-red-black-...) is at most twice the shortest (all black), because between any two black nodes along a path you can have at most one red.

Definition — Height bound

Let bh = black-height of the root. Then:

- Shortest path = bh (all black).
- Longest path $\leq 2\cdot$ bh (alternating red-black).

A red-black tree with n nodes has $h \leq 2\log_2(n+1)$, i.e., at most twice a perfectly balanced tree's height. Still $O(\log n)$.

Compare:

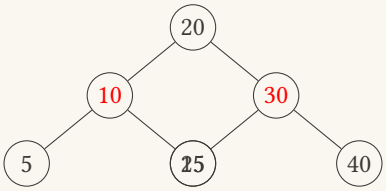
Structure	Worst-case height	Typical overhead per node
Perfect BBST	$\log_2 n$	impossible to maintain
AVL	$\approx 1.44\log_2 n$	integer height field (4 bytes)
Red-black	$\leq 2\log_2 n$	1 color bit (usually stolen from pointer)
Raw BST (no balance)	$n-1$	none

Red-black trees allow roughly 1.4× more height than AVL, which sounds worse. In practice, red-black wins for the following reasons:

- The color bit costs less than AVL's integer height field.
- Red-black needs fewer rotations per update in practice. Deletion in particular: AVL can cascade $O(\log n)$ rotations, red-black is bounded by 3.
- Red-black's amortized cost is lower on mixed workloads; AVL's strict balance costs more constant work to maintain.

Example: validating a red-black tree

Is this a valid red-black tree? Red nodes are in **red** text; black nodes plain. Assume every null is a black sentinel.



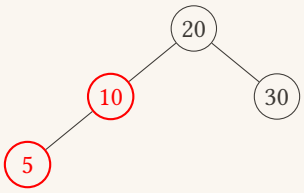
Check each rule:

1. Every node colored – yes.

2. Root (20) is black – yes.
3. No red-red: 10 and 30 are red; their children (5, 15, 25, 40) are black – OK.
4. Nulls are black leaves – by convention.
5. Black-heights: from 20 to any null sentinel under 5, 15, 25, or 40, we pass through $20, 5/15/25/40$, sentinel = 3 black nodes. Same on every path – OK.

All rules hold. Valid red-black tree.

Counterexample: a red-red violation



Node 10 is red, and its child 5 is also red. Rule 3 violated. Not a valid red-black tree. Insertion or deletion code has to fix this kind of situation with recolorings and rotations – that's what sections 9.6 and 9.7 will cover.

The bit-stealing trick

In practice, the color bit is not stored as a separate byte. Every pointer on a modern architecture is 8-byte aligned, which means the low three bits of every node pointer are zero. Implementations "steal" one of those bits to encode color: `uintptr_t(parent) | 1` means red, `uintptr_t(parent) & ~1` means black. Saves a full word per node and is standard technique in `std::_Rb_tree_node`.

This isn't required for correctness – you can use a `bool isRed` field – but it shows how mature implementations pay zero storage overhead for the color state.

Key idea

Red-black trees are a compromise: slightly looser balance invariant than AVL, but much more forgiving during rebalancing. The five rules look fiddly but are actually the minimal set that forces $O(\log n)$ height with the weakest possible constraints. AVL's "differ by at most 1" is tighter but more expensive to maintain. Red-black is the industry-standard choice because its looseness costs almost nothing in practice and its rebalancing code is (arguably) more tractable once you learn the cases.

Looking ahead

Section 9.6 will cover red-black **insertion**: attach a new node (always colored *red* initially), then fix any red-red violation by recoloring or rotating. Section 9.7 will cover red-black **removal**: harder, because the deleted node's "blackness" can propagate upward as a "double-black" that needs careful resolution. Both operations end up with a small number of named cases – six for insert, eight for delete – which is why people find red-black trees intimidating. The good news: every case is a constant-work local fix.

Gotcha

Don't try to memorize the rules without understanding why they hold together. Rule 3 is the "no clusters" rule; rule 5 is the "balanced depth" rule; rules 1, 2, and 4 are administrative. If you forget a rule during a bug hunt, re-derive it: if rule 5 could be violated freely, a tree could be arbitrarily tall on one side with no compensating reds; if rule 3 could be violated, you could stack many reds along one path and bypass rule 5 trivially.

6 9.6 Red-Black Rotations

Red-black rotations are *the same operation* as AVL rotations: rewire three pointers to preserve BST ordering while reshaping a local subtree. The implementation differs only in what auxiliary state is maintained (color instead of height) and what triggers a rotation (a red-red violation or a black-height violation, not a ± 2 balance factor).

This section states the primitives. Sections 9.7 uses them to implement red-black insert (and would cover remove).

Definition — Red-black rotation

A rotation on a red-black tree is mechanically identical to an AVL rotation:

- **Left rotation at** X requires X's right child to be non-null. X's right child becomes the subtree's new root. (Colors are *not* changed by the rotation itself – they're a separate bookkeeping step.)
- **Right rotation at** X requires X's left child to be non-null. Symmetric.

No colors are rotated; they stay on their original nodes. This is subtle: in a rotation the node identities don't change, only the parent/child edges do. So if X was red before a left rotation, X is still red after; only its position in the tree moved.

Utility helpers

The helper signatures mirror AVL's (section 9.2) minus the `avlUpdateHeight` call. Red-black doesn't cache a per-node height; it maintains color instead, which changes via explicit recolor operations (section 9.7), not automatically from structural changes.

```
enum class Color { Red, Black };

template <typename T>
struct RBNode {
    T        data;
    Color    color  = Color::Red;      // new nodes default red (see 9.7)
    RBNode*  left   = nullptr;
    RBNode*  right  = nullptr;
    RBNode*  parent = nullptr;
};

template <typename T>
void rbSetChild(RBNode<T>* parent, bool isLeft, RBNode<T>* child) {
    if (isLeft) parent->left = child;
    else        parent->right = child;
    if (child != nullptr) child->parent = parent;
}
```

```
template <typename T>
void rbReplaceChild(RBNode<T>* parent,
                    RBNode<T>* currentChild,
                    RBNode<T>* newChild) {
    if (parent->left == currentChild)  rbSetChild(parent, true,  newChild);
    else                               rbSetChild(parent, false, newChild);
}
```

No height recomputation. No balance-factor check. Just pointer rewiring.

Left rotation algorithm

```
template <typename T>
void rbRotateLeft(RBNode<T>*& root, RBNode<T>* X) {
    RBNode<T>* Y  = X->right;        // must be non-null
    RBNode<T>* YL = Y->left;         // may be null

    if (X->parent != nullptr) rbReplaceChild(X->parent, X, Y);
    else                      { root = Y; Y->parent = nullptr; }

    rbSetChild(Y, true,  X);         // Y's left becomes X
    rbSetChild(X, false, YL);        // X's right becomes Y's old left
}
```

Right rotation algorithm

The mirror image:

```
template <typename T>
void rbRotateRight(RBNode<T>*& root, RBNode<T>* X) {
    RBNode<T>* Y  = X->left;         // must be non-null
    RBNode<T>* YR = Y->right;        // may be null

    if (X->parent != nullptr) rbReplaceChild(X->parent, X, Y);
    else                      { root = Y; Y->parent = nullptr; }

    rbSetChild(Y, false, X);
    rbSetChild(X, true,  YR);
}
```

Side-by-side with AVL rotation

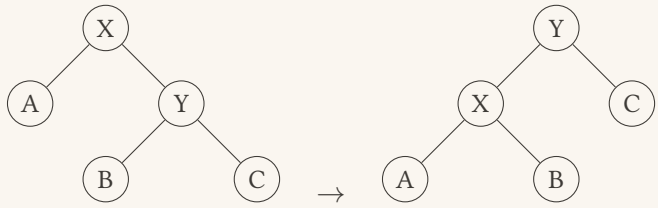
	AVL	Red-Black
Pointer rewires	3	3
Update cached field	height(X), height(Y)	none during rotate
Triggered by	$\mathrm{bf}(X) \in \{-2, +2\}$	red-red or black-height violation
Preserves BST ordering	yes	yes
Preserves balance	no – it *fixes* balance	no – it *helps fix* balance along with recoloring

The shape-changing part is identical. Red-black's twist is that rotations alone don't restore the invariant –

you need **rotations + recolorings** together. Section 9.7 will show how those two primitives compose.

Example: a rotation as standalone visualization

Before left-rotate at X:



In-order sequence before: A, X, B, Y, C. After: A, X, B, Y, C. Identical. That's the invariant rotations preserve.

When rotations alone aren't enough

A single rotation on a red-black tree typically *does not* restore all five invariants. For example, a red node rotated to the top of a subtree might violate rule 2 (root must be black) or rule 3 (no red-red). That's why red-black insert and remove always combine rotations with explicit recoloring steps.

The cleanest way to think about it: rotations change *structure*, recolorings change *color state*, and the balanced-search-tree operations use both together to fix whatever rule was broken. Section 9.7 makes the dance explicit.

Key idea

It's worth internalizing: every self-balancing BST ever invented (AVL, red-black, splay, weight-balanced, scapegoat, treap, AA-tree, 2-3-4 tree) uses rotations as its shape-changing primitive. The differences are only in the invariant being maintained and the trigger conditions for rotation. "Rotations preserve BST ordering" is the single fact you need to believe in; everything else is bookkeeping.

Gotcha

Never conditionally skip a rotation "because the tree looks OK." The insert/remove algorithms decide when to rotate based on the color configuration, not on eyeball balance. If you add a conditional like "only rotate if the imbalance is large enough," you'll corrupt black-heights and the tree will silently stop being a valid red-black tree.

Aside

Production red-black implementations (the `std::map` backing tree, Java's `TreeMap`, Linux's `rbtree.c`) all use some variant of the helpers shown here. They inline the rotations into the insert/delete case code for speed, but the primitive is the same 3-pointer rewire with parent-pointer maintenance. Reading one production implementation once – Linux's is the cleanest – is one of the best ways to internalize the operations.

7 9.7 Red-Black Insertion

Red-black insertion has a reputation for being intimidating. The reputation is partially deserved – there are more cases than in AVL – but the logic breaks into a handful of symmetric pairs once you see the pattern. This section walks through the full algorithm and the five cases that can arise.

Definition — Red-black insert, high level

1. **BST insert** the new node at the correct leaf position.
2. **Color it red.** New nodes are always red initially. This preserves black-height (rule 5) but may create a red-red violation (rule 3) if the parent is also red.
3. **Fix up** any red-red violation by recoloring and/or rotating. The fix-up walks from the new node toward the root, possibly recursing once per pair of levels.

Why color new nodes red?

A color choice has to be made. If we colored new nodes black, rule 5 (equal black-height on every path) would immediately break – the path through the new node now has one more black than the rest. Fixing that violation is hard; it cascades globally.

Red breaks only rule 3 (no red child of a red parent), and rule 3 is local – you can see if it's violated by checking the new node's parent. If the parent is black, no violation, done. If the parent is red, local fix with one of five named cases. Much easier.

Terminology: parent, uncle, grandparent

Definition — Family relationships

For a node n:

- **Parent** = n's parent.
- **Grandparent** = n's parent's parent (may be null).
- **Uncle** = the grandparent's *other* child – the sibling of n's parent (may be null).

The color of the uncle is the key discriminator in the insertion fix-up. A red uncle lets us "push the violation up" via recoloring; a black (or null) uncle requires a rotation.

```
template <typename T>
RBNode<T>* rbGrandparent(RBNode<T>* n) {
    return (n->parent != nullptr) ? n->parent->parent : nullptr;
}

template <typename T>
RBNode<T>* rbUncle(RBNode<T>* n) {
    RBNode<T>* g = rbGrandparent(n);
    if (g == nullptr) return nullptr;
    return (g->left == n->parent) ? g->right : g->left;
}
```

The five cases

Let n = the newly-inserted node, p = n's parent, g = n's grandparent, u = n's uncle.

Case 1: n is the root. Just color it black. Done. (Rule 2.)

Case 2: p is black. No red-red violation. Done.

Case 3: p is red and u is red. Recolor: p and u both become black, g becomes red. Now black-heights are

unchanged below g, but g might now violate rule 3 with its own parent. *Recursively apply fix-up to g.*

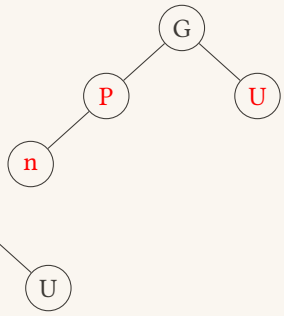
Case 4: p is red, u is black (or null), n is on the "inside." *Inside* means n is a right child and p is a left child, or vice versa. Rotate at p to straighten the zig-zag into a zig-zig. Then fall through into case 5.

Case 5: p is red, u is black (or null), n is on the "outside." *Outside* means n and p are on the same side (both lefts, or both rights). Recolor p black and g red. Rotate at g (right if p was left, left if p was right). Done.

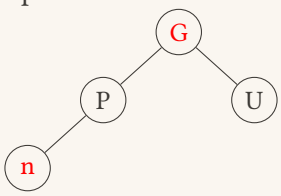
The pattern: cases 3 and 5 recolor; cases 4 and 5 rotate; case 3 recurses; cases 4 and 5 terminate in $O(1)$.

Case 3 in detail: uncle is red

Before: parent P red, uncle U red, grandparent G black.



$\rightarrow$ *After recolor:*

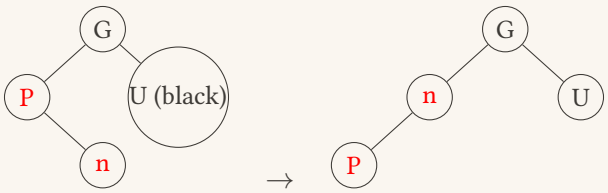


No structure changed; only colors. But G is now red, which may conflict with G's parent. Recurse the fix-up on G. Black-height from any ancestor of G is preserved because G's path had one black (itself) replaced by a red (itself) but both paths through P and U gained a black (those nodes became black), so the net along every path stayed the same.

Cases 4 and 5: uncle is black

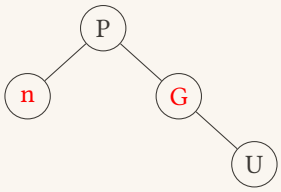
This is where rotations come in. If the uncle is black, recoloring alone would break rule 5 – the uncle's path was already black-height balanced, and changing colors without a rotation would shift the count.

Case 4 (zig-zag $\rightarrow$ zig-zig): suppose n is the right child of p, and p is the left child of g. Left-rotate at p; this swings n into p's old position with p as n's left child. Now n and its new left child (p) are both "on the left" of g – we've transformed into case 5. Relabel: what was "n" we now call "p," and fall through.



Now apply case 5.

Case 5 (zig-zig $\rightarrow$ fixed): Recolor P black, G red, then rotate right at G:



Now P is black (satisfies root or whatever parent G had), n and G are red (but neither has a red parent), and the black-height through every path is the same as before the insertion. Done in $O(1)$.

The full fix-up loop

```
template <typename T>
void rbBalance(RBNode<T>*& root, RBNode<T>* n) {
    while (true) {
        if (n->parent == nullptr) {             // case 1: n is root
            n->color = Color::Black;
            return;
        }
        if (n->parent->color == Color::Black)   // case 2
            return;

        RBNode<T>* p = n->parent;
        RBNode<T>* g = rbGrandparent(n);
        RBNode<T>* u = rbUncle(n);

        if (u != nullptr && u->color == Color::Red) {  // case 3
            p->color = Color::Black;
            u->color = Color::Black;
            g->color = Color::Red;
            n = g;                              // recurse upward
            continue;
        }

        // Cases 4/5: uncle is black (or null).
        // Case 4: straighten zig-zag into zig-zig.
        if (n == p->right && p == g->left) {
            rbRotateLeft(root, p);
            n = n->left;                        // which is the old p
        } else if (n == p->left && p == g->right) {
            rbRotateRight(root, p);
            n = n->right;                       // which is the old p
        }

        // Case 5: zig-zig fix.
        p = n->parent;
        g = rbGrandparent(n);
        p->color = Color::Black;
        g->color = Color::Red;
        if (n == p->left) rbRotateRight(root, g);
        else              rbRotateLeft(root, g);
        return;                                 // O(1) termination
    }
}
```

```
template <typename T>
void rbInsert(RBNode<T>*& root, RBNode<T>* node) {
    // Phase 1: BST insert. (Identical to chapter 6 insert; omitted.)
    bstInsert(root, node);
    node->color = Color::Red;
    rbBalance(root, node);
}
```

Cost

Phase	Cost
BST descent	$O(\log n)$
Fix-up loop	$O(\log n)$ iterations in the worst case (case 3 recurses)
Rotations per insertion	At most 2 (from case 4/5, one shot)
Recolorings per insertion	Up to $O(\log n)$ (case 3 cascade)
Total	$\mathbf{O(\log n)}$

Insert is always $O(\log n)$ and – notably – does at most *two* rotations, regardless of how many recolorings happen. That low rotation count (compared to AVL's worst case) is part of why red-black is preferred in practice: rotations are more expensive than recolorings (three pointer writes vs. one bit flip), and capping rotations at 2 per insert is a solid guarantee.

Key idea

The case-3 recursion is "the violation walks up the tree" – each level is fixed by demoting the grandparent to red, which may re-create a red-red with great-grandparent, and so on. But each step only touches three nodes' colors; no rotations yet. Only when we hit a black uncle do we commit to a rotation, and that rotation terminates the whole cascade. The recoloring is the workhorse; the rotations are the cleanup.

A worked insertion sequence

Start empty. Insert 22, 11, 33, 55, 44:

- `insert(22)`: root, colored black. Tree: {22(B)}.
- `insert(11)`: BST puts it left of 22, colored red. Parent black, case 2. Tree: {22(B), 11(R)}.
- `insert(33)`: right of 22, red. Parent black, case 2. Tree: {22(B), 11(R), 33(R)}.
- `insert(55)`: right of 33, red. Parent 33 is red, uncle 11 is red – case 3. Recolor: 11(B), 33(B), 22(R). Recurse at 22. 22 is root, case 1 forces it back to black. Tree: {22(B), 11(B), 33(B), 55(R)}.
- `insert(44)`: left of 55, red. Parent 55 red, uncle = right child of 33 = null (black). 44 is left child of 55, 55 is right child of 33 – case 4 (zig-zag). Right-rotate at 55: 44 becomes 33's right child, 55 becomes 44's right child. Then case 5: recolor 44(B), 33(R). Left-rotate at 33. Done.

The whole sequence involves one cascading recolor and one case-4-plus-5 double-rotation. Total for five insertions: at most 2 rotations and a handful of color flips. Tree stays $O(\log n)$ tall throughout.

Gotcha

Case 4 and case 5 are *sequential*, not alternatives. Case 4 transforms the imbalance into a shape that case 5 can finish. Don't write them as an `if/else` that picks one or the other – write case 4 as a precondition transform that falls through into case 5. The loop structure should be "optionally do case 4, then always do case 5."

Aside

Red-black *removal* follows the same spirit – BST remove first, then fix any invariant violation – but with more cases because removing a black node breaks rule 5 directly. That's the subject of the next section.

8 9.8 Red-Black Tree: Removal

Red-black removal is the most intricate operation in any red-black implementation. Insertion has five cases, rotations are a few lines – removal has six named cases on top of the BST removal logic. This section walks through all of it, but keep the big picture in mind while the details fly by:

Key idea

The whole removal story, one paragraph.

1. BST-remove the node. But first, if it's internal, swap with its predecessor so we're always deleting a node with at most one child.
2. Removing a *red* node is free – black-heights don't change, no rule is violated. Done.
3. Removing a *black* node drops the black count along one path by 1, violating rule 5 (equal black-heights). We have to fix this *before* we remove the node.
4. The fix is a cascade of six cases that massage the tree so the node becomes removable without violating any invariant – sometimes by recoloring, sometimes by rotating, sometimes by pushing the problem one level up.

Step 1: High-level remove

The entry point is `rbRemove(tree, key)`, which is just BST search + dispatch to node-removal:

```
void rbRemove(RBTree& t, int key) {
    RBNode* n = bstSearch(t.root, key);
    if (n) rbRemoveNode(t, n);
}
```

`rbRemoveNode` handles the classic BST "two-children" reduction: if `n` has both children, copy the predecessor's key into `n` and recursively remove the predecessor node instead. Predecessors are leaves or have only a left child, so we always end up removing a node with ≤ 1 child:

```
void rbRemoveNode(RBTree& t, RBNode* n) {
    if (n->left && n->right) {
        RBNode* pred = rbPredecessor(n);
        int predKey = pred->key;
        rbRemoveNode(t, pred);   // recurse on a <= 1-child node
        n->key = predKey;
```

```
        return;
    }
    if (n->color == Color::Black)
        rbPrepareForRemoval(t, n);   // fix black-height BEFORE removing
    bstRemove(t, n->key);            // ordinary BST splice-out
}

RBNode* rbPredecessor(RBNode* n) {
    n = n->left;
    while (n->right) n = n->right;
    return n;
}
```

Gotcha

Order matters. `rbPrepareForRemoval` runs *before* `bstRemove`. We want to rebalance while the tree still has `n` in place, so we can identify its sibling, parent, and nephews. After the BST splice, that context is gone.

Why red is easy, black is hard

Removing a node means splicing out `n` and replacing it with its (single or null) child. Two rules could break:

- **Rule 3 (no two reds in a row).** Removing a node can't *create* a red-red pair as long as we replace with the correct child. Insertion worries about this; removal doesn't.
- **Rule 5 (equal black-heights).** Every root-to-null path must have the same count of black nodes. Removing a *black* node decrements that count along one path.

If `n` is red, the black count doesn't change, and rule 3 is safe because `n`'s child must have been black (rule 3). Nothing to do.

If `n` is black, rule 5 breaks. Classical textbooks solve this by imagining `n`'s replacement carries an extra "double-black" token, and the fix-up pushes that token around until it reaches the root or cancels against a red. The textbook here takes an equivalent but more operational view: classify the local neighborhood into six cases and rewrite it until removal is safe.

Utility functions

Four helpers make the case logic readable. Critically, **a null pointer counts as a black node** in all of them – matching rule 2 (null leaves are black):

```
RBNode* rbSibling(RBNode* n) {
    if (!n->parent) return nullptr;
    return (n == n->parent->left) ? n->parent->right : n->parent->left;
}

bool rbIsRed(RBNode* n) {
    return n && n->color == Color::Red;
}

bool rbIsBlack(RBNode* n) {
```

```
    return !n || n->color == Color::Black;   // null is black
}

bool rbBothChildrenBlack(RBNode* n) {
    return rbIsBlack(n->left) && rbIsBlack(n->right);
}
```

Step 2: Prepare-for-removal, the six cases

The prepare-for-removal routine classifies the local neighborhood of n (its color, parent, sibling, and sibling's children) and applies one of six rewrites. Cases 1, 3, 4 are terminal – once they fire, prep is done. Cases 2, 5, 6 are *transitional* – they massage the tree and fall through to later cases.

Example

What each case is really trying to do.

- **Case 1** "We're already OK." Node is red, or we're at the root – nothing to fix.
- **Case 2** "Sibling is red – make it black first." Rotate to put a black sibling in place so cases 3–6 can run.
- **Case 3** "Whole neighborhood is black – push the deficit up." Recolor sibling red, recurse on the parent. The subtree is now internally balanced, but the parent has lost a black from above; that's the parent's problem now.
- **Case 4** "Parent is red, sibling branch is all black – swap colors locally." Swap parent's red to sibling, parent becomes black. Done in $O(1)$.
- **Case 5** "Zig-zag sibling – straighten it." Rotate on the sibling so we can finish with case 6's big rotation.
- **Case 6** "Final rotation." Rotate at the parent, recolor. This is where the deficit is permanently absorbed.

The driver

```
void rbPrepareForRemoval(RBTree& t, RBNode* n) {
    if (rbTryCase1(n)) return;

    RBNode* s = rbSibling(n);
    if (rbTryCase2(t, n, s)) s = rbSibling(n);
    if (rbTryCase3(t, n, s)) return;
    if (rbTryCase4(n, s))    return;
    if (rbTryCase5(t, n, s)) s = rbSibling(n);
    if (rbTryCase6(t, n, s)) s = rbSibling(n);

    // Fall-through: the final rotation (always runs if we got here).
    s->color = n->parent->color;
    n->parent->color = Color::Black;
    if (n == n->parent->left) {
        s->right->color = Color::Black;
        rbRotateLeft(t, n->parent);
    } else {
        s->left->color = Color::Black;
        rbRotateRight(t, n->parent);
```

```
    }
}
```

Note the structure: after case 2, the sibling may have changed (it rotated out), so we refetch. Same after cases 5 and 6. Cases 3 and 4 `return` immediately. Cases 5 and 6 *prepare* the configuration for the final fall-through rotation.

Case 1 – trivially removable

```
bool rbTryCase1(RBNode* n) {
    return (n->color == Color::Red) || (n->parent == nullptr);
}
```

A red node's removal doesn't disturb black-heights. A root node's removal – after the BST splice reduces it to a trivial case – is also fine (any single remaining child becomes the new root, will be recolored black implicitly by the fix-up of its children). No action required.

Case 2 – red sibling

```
bool rbTryCase2(RBTree& t, RBNode* n, RBNode* s) {
    if (!rbIsRed(s)) return false;
    n->parent->color = Color::Red;
    s->color = Color::Black;
    if (n == n->parent->left) rbRotateLeft(t, n->parent);
    else                      rbRotateRight(t, n->parent);
    return true;
}
```

If the sibling is red, the parent must be black (rule 3), and the sibling's two children must be black (rule 3 again). Rotate at the parent to promote the sibling up, recolor so the old parent is red and the old sibling is black. `n` now has a *black* sibling (which is one of the former sibling's children) – that's the precondition for cases 3–6, which do the real work.

Aside

Case 2 doesn't solve anything by itself – it normalizes the tree so the harder cases can match. That's why it falls through.

Case 3 – push the deficit up

```
bool rbTryCase3(RBTree& t, RBNode* n, RBNode* s) {
    if (n->parent->color != Color::Black) return false;
    if (!rbBothChildrenBlack(s))          return false;
    s->color = Color::Red;
    rbPrepareForRemoval(t, n->parent);   // recurse one level up
    return true;
}
```

Parent black, sibling black, sibling's children black. Recolor the sibling red – that drops the black count on the sibling's side by 1, matching the drop we're about to cause on n's side. Now the subtree rooted at the parent has consistent black-heights internally, but the parent as a whole has *one fewer black* on all paths through it than before. That's the same problem we started with, one level higher – so we recurse on the parent.

Key idea

Case 3 is where the $O(\log n)$ cost of removal comes from. The other cases are constant-time terminal fixes; only case 3 can chain up the tree, and it runs at most $h = O(\log n)$ times before hitting the root (case 1).

Case 4 – swap colors

```
bool rbTryCase4(RBNode* n, RBNode* s) {
    if (n->parent->color != Color::Red) return false;
    if (!rbBothChildrenBlack(s))        return false;
    n->parent->color = Color::Black;
    s->color = Color::Red;
    return true;
}
```

Parent red, sibling branch all black. Swap: parent becomes black, sibling becomes red. On n's path, we lose a black (from removing n) but gain one (parent flipped to black) – net zero. On the sibling's path, we lose one (sibling flipped to red) but gain one (parent flipped to black) – net zero. Rule 3 holds because the sibling's children were already black. Done.

Case 5 – straighten zig-zag (mirror of insertion case 4)

```
bool rbTryCase5(RBTree& t, RBNode* n, RBNode* s) {
    if (!rbIsRed(s->left))             return false;
    if (!rbIsBlack(s->right))          return false;
    if (n != n->parent->left)          return false;
    s->color = Color::Red;
    s->left->color = Color::Black;
    rbRotateRight(t, s);
    return true;
}
```

This preps for case 6. If n is a left child and the sibling's "outer" child (sibling's right) is black while its "inner" child is red, a rotation at the sibling moves the red to the outer position – now case 6 (or the fall-through) can do its single final rotation.

Case 6 – mirror of case 5

```
bool rbTryCase6(RBTree& t, RBNode* n, RBNode* s) {
    if (!rbIsBlack(s->left))           return false;
    if (!rbIsRed(s->right))            return false;
    if (n != n->parent->right)         return false;
    s->color = Color::Red;
```

```
    s->right->color = Color::Black;
    rbRotateLeft(t, s);
    return true;
}
```

Same as case 5 but for when n is a right child. Prepares for the fall-through rotation.

The fall-through rotation

After case 5 or 6 (or if neither applied), the sibling has a red child in the "outer" position. The driver's tail runs:

- Sibling inherits the parent's color.
- Parent becomes black.
- The outer red child of the sibling becomes black.
- Rotate at the parent (left if n was left child, right otherwise).

The effect: the sibling becomes the new subtree root, painted the old parent's color (so external black count is preserved). The parent goes down to n's side, now black (compensating for n's upcoming removal). The sibling's outer red child goes to the opposite side, now black (compensating for the shift on that side). All black-heights are restored – and n can now be BST-removed without any violation.

Why this many cases?

Key idea

Red-black removal has six cases because the local neighborhood can be in six structurally distinct shapes, each needing a different rewrite to restore rule 5 without breaking rule 3. Insertion had only five cases because the only inserted node is always red and always a leaf – removal has to cope with arbitrary existing structure around the deletion point.

Gotcha

The textbook treatment varies. CLRS presents the same algorithm as four cases by merging symmetric pairs; the presentation here splits the fall-through rotation into two steps (cases 5/6 + the tail) where CLRS fuses them. The operations are identical; only the bookkeeping differs. If you cross-reference another source, expect the case numbering to shift.

Cost analysis

- **Find the node**: $O(\log n)$ BST search.
- **Two-children reduction**: at most one recursive descent to the predecessor, another $O(\log n)$.
- **Prepare-for-removal**: at most 3 rotations (case 2 + case 5 + fall-through, or case 2 + case 6 + fall-through). Case 3 can recurse up the tree $O(\log n)$ times, but each iteration is $O(1)$ recoloring. So the total fix-up work is $O(\log n)$.
- **Total**: $O(\log n)$ time, $O(\log n)$ stack depth (can be rewritten iteratively with parent pointers).

Aside

AVL vs. red-black removal. AVL removal can cascade up to $O(\log n)$ rotations; red-black removal does at most 3 rotations (the rest is just recoloring). That's the classic reason `std::map` chose red-black: insertion/removal are both cheaper in rotation count, and rotations involve actual pointer writes, while recoloring is just a bit flip. For workloads heavy on modifications, the constant-factor win matters.

Putting it all together

The complete removal interface:

```
void rbRemove(RBTree& t, int key) {
    RBNode* n = bstSearch(t.root, key);
    if (n) rbRemoveNode(t, n);
}

void rbRemoveNode(RBTree& t, RBNode* n) {
    if (n->left && n->right) {
        RBNode* pred = rbPredecessor(n);
        int k = pred->key;
        rbRemoveNode(t, pred);
        n->key = k;
        return;
    }
    if (n->color == Color::Black)
        rbPrepareForRemoval(t, n);
    bstRemove(t, n->key);
}
```

Key idea

The mental model to keep. A red-black tree is a BST with bookkeeping. Every modifying operation is (1) do the BST thing, (2) fix the coloring/height invariants locally, (3) if local fix is impossible, propagate one level up. Insertion propagates up with *uncle-red* recoloring; removal propagates up with *case 3* recoloring. Both walk at most $O(\log n)$ levels, bounded by the tree height, which rule 5 guarantees is $O(\log n)$.